SLE-4: Architecture Decision Record (ADR)

Graph Search Profiling System Using Breadth-First Search and Depth-First Search

PRN: 26UAM314

Name: Archana Shankar Dropatkar

Division: A

Date: 11/10/2026

Status: Accepted

==========

PART A - Architecture Decision Record (ADR)

==========

ADR-01: Selection of Breadth-First Search (BFS) for Level-by-Level Graph Traversal

1. Context

The project focuses on a Graph Search Profiling System developed to implement, analyse, and compare the performance of Breadth-First Search (BFS) and Depth-First Search (DFS) on graph-based data. In modern software applications and AI systems, efficient graph traversal plays an important role in exploring connected nodes, locating target elements, and understanding the behaviour of search algorithms.

During graph traversal and goal-search operations, nodes must be explored systematically while avoiding repeated visits and unnecessary processing. BFS explores neighbouring nodes level by level using a queue, whereas DFS follows a path deeply before backtracking to explore other branches. As the size and complexity of a graph increase, the traversal strategy and data structures used can affect execution time and memory requirements. Therefore, an architectural evaluation is necessary to compare both approaches, analyse their profiling results, and identify the most suitable algorithm for different graph-search requirements.

2. Decision

We decided to implement **Breadth-First Search (BFS)** as the preferred approach for level-by-level graph traversal within our system. This decision is intended to support systematic node exploration, simplify the identification of the shortest path in an unweighted graph, and provide a clear basis for evaluating search performance. Depth-First Search (DFS) is retained as an alternative algorithm for comparison and deep graph exploration.

The BFS function begins with the starting node and stores discovered nodes in a queue. It removes nodes from the front of the queue, checks whether the current node matches the target, and adds unvisited neighbouring nodes to the queue. This process continues until the target is found or the queue becomes empty, indicating that no reachable target was discovered.

3. Alternatives Considered

Alternative 1: Depth-First Search (DFS)

- **Description:** Explores a graph by following one branch as deeply as possible before backtracking to visit other unexplored nodes.
- **Reason for Not Selecting as the Primary Approach:** DFS does not guarantee the shortest path in an unweighted graph. Its traversal depends on the order in which neighbouring nodes are visited, making it less suitable when the main requirement is to find the shortest path by number of edges.

Alternative 2: Using Only One Search Algorithm

- **Description:** Implements either BFS or DFS as the only graph traversal method, without maintaining a second algorithm for comparison.
- **Reason for Rejection:** Using only one algorithm limits the ability to compare traversal behaviour, execution time, and node expansion. Keeping both algorithms provides a better understanding of their performance under the same graph conditions.

Alternative 3: Breadth-First Search (BFS) – Selected

- **Description:** BFS is a graph traversal algorithm that explores nodes level by level using a queue. It visits the neighbouring nodes of the current node before proceeding to the next level.
- **Reason for Selection:** BFS is suitable for systematic graph exploration and guarantees the shortest path in an unweighted graph when implemented correctly. Its queue-based approach also makes the traversal process easier to understand, test, and compare with DFS during performance profiling.

4. Rationale / Justification

Breadth-First Search (BFS) provides a systematic method for exploring graph nodes by visiting neighbouring nodes level by level. By using a queue, the algorithm processes nodes in the order in which they are discovered. This approach is particularly useful for finding the shortest path in an unweighted graph and comparing graph traversal performance.

In our implementation, BFS maintains a queue and a visited set to manage node exploration:

queue = deque([start])

visited = set()

The algorithm removes a node from the front of the queue, checks whether it is the target, and adds its unvisited neighbours to the queue. This process repeats until the target is found or no nodes remain to be explored.

This architectural decision is supported by the profiling work carried out in SLE-2, where BFS and DFS were implemented and evaluated using Python scripts. The recorded comparison output showed the following results for the tested graph:

- **Execution Time:** BFS recorded approximately 12.6392 ms, whereas DFS recorded approximately 16.4543 ms in the recorded test run.
- **Nodes Expanded:** Both BFS and DFS expanded 6 nodes in the recorded comparison, providing a basis for examining their execution behaviour on the same graph.
- **Performance Analysis:** The recorded result indicates that BFS completed faster than DFS in this particular test. However, execution times may change depending on graph structure, traversal order, implementation details, and system conditions. Therefore, these measurements should not be interpreted as proof that BFS is always faster.
- **Algorithmic Complexity:** With an adjacency-list representation, BFS and DFS generally have a time complexity of $O(V + E)$, where Vrepresents the number of vertices and Erepresents the number of edges. BFS can require additional memory for its queue, particularly when many nodes exist at the same level.

This decision is related to the SLE-3 C4 architecture, which divides the graph-search system into modules for input, graph data, search, visited nodes, path processing, and output. This modular design makes the system easier to understand, maintain, and test.

By connecting SLE-2 implementation, SLE-3 architecture, and SLE-4 decision documentation, the project shows how algorithm selection can be justified using code, profiling results, and system requirements.

5. Consequences

Positive Consequences

- **Systematic Graph Traversal:** BFS explores nodes level by level, making it suitable for finding the shortest path in an unweighted graph.
- **Clear Search Behaviour:** The queue-based approach provides an organised traversal process and helps avoid revisiting nodes when a visited set is maintained correctly.
- **Modular System Architecture:** BFS and DFS can be maintained as separate search algorithms within the SLE-3 C4 architecture, making the system easier to understand, test, and improve.
- **Supports Performance Evaluation:** The decision is supported by the timing measurements and node-expansion results recorded during SLE-2 profiling, allowing both algorithms to be compared under the same test conditions.

Negative Consequences / Trade-offs

- **Higher Memory Requirements:** BFS may require additional memory to store the queue of discovered nodes, especially when the graph contains many nodes at the same level.
- **Dependence on Graph Structure:** The execution time and number of nodes explored depend on the graph connections, starting node, target node, and order of neighbouring nodes.

- **Limited Suitability for Weighted Graphs:** Standard BFS does not guarantee the minimum-cost path when edges have different weights. A suitable weighted-graph algorithm is required for such cases.

==========

PART B - Short SLE Journey

==========

SLE-1: Basic AI Agent

I developed a basic AI agent using Python and organised the code in GitHub. This helped me learn AI workflows, programming, and project documentation.

SLE-2: BFS and DFS Profiling

I implemented BFS and DFS algorithms and compared their performance using Python timing and profiling tools. This improved my understanding of graph traversal and algorithm analysis.

SLE-3: C4 Architecture Design

I created C4 diagrams at Context, Container, Component, and Code levels to explain the structure and interaction of my graph-search system modules.

SLE-4: Architecture Decision Record

I prepared an ADR explaining why BFS is suitable for level-by-level traversal and shortest-path finding in unweighted graphs, while DFS is retained for comparison and deep exploration.

with software architecture principles and prepared me to explain the project during the final viva.

==========

PART C - AI Contribution Note

==========

- **AI Tools Used:** ChatGPT, GitHub Copilot, and Gemini, where applicable.
- **How AI Assisted:** AI tools helped me understand BFS and DFS concepts, organise the technical documentation, improve explanations of algorithm complexity, interpret profiling results, and structure the Architecture Decision Record (ADR).
- **My Individual Contribution:** I worked on the Python project environment, implemented and executed the BFS and DFS programs, examined the profiling output, documented the observed performance results, organised the GitHub repository, and prepared the C4 architecture diagrams and architectural explanations. I also reviewed

the final ADR content and prepared the viva notes to explain the design decisions and their trade-offs.

==========

PART D - Viva Preparation Notes & Defense Guide

==========

1. Main Architectural Decision: We selected Breadth-First Search (BFS) as the preferred algorithm for level-by-level graph traversal and finding the shortest path in an unweighted graph. Depth-First Search (DFS) is retained for deep exploration and performance comparison.

2. Profiling Results (SLE-2): The recorded profiling output showed that BFS completed in approximately 12.6392 ms, while DFS took approximately 16.4543 ms. Both algorithms expanded 6 nodes in that test. These measurements help compare their performance under the tested conditions.

3. Connection with SLE-3 Architecture: The C4 architecture represents the graph-search system through separate responsibilities, including graph input, search execution, visited-node management, path processing, and output generation. This modular design makes the system easier to understand and maintain.

4. Algorithm Scalability: BFS and DFS generally have a time complexity of $O(V + E)$when an adjacency-list representation is used, where $V$is the number of vertices and $E$is the number of edges. Their practical performance and memory requirements depend on the graph structure and implementation.

5. Implementation and Verification: The BFS and DFS programs, profiling scripts, and architecture diagrams are maintained in the project repository. Their outputs can be checked by running the programs locally and comparing the generated results with the documented observations.

==========

Conclusion

==========

The key architectural decision in this project was to select Breadth-First Search (BFS) as the preferred method for level-by-level graph traversal and shortest-path discovery in unweighted graphs, while retaining Depth-First Search (DFS) for comparison and deeper exploration. This decision is reflected in the BFS and DFS implementations developed during SLE-2 and is connected to the modular structure represented in the SLE-3 C4 architecture.

The profiling results collected in SLE-2 provided a practical basis for comparing the execution behaviour of both algorithms under the tested conditions. SLE-4 brings together this implementation, profiling, and architectural work in a structured Architecture Decision Record (ADR). Overall, the project demonstrates how selecting an appropriate algorithm, evaluating its performance, and documenting architectural trade-offs can contribute to a clear, maintainable, and well-organised software system.